

CREWMATE

7-Layer Decoupled Enterprise Multi-Agent Platform (GEAP)

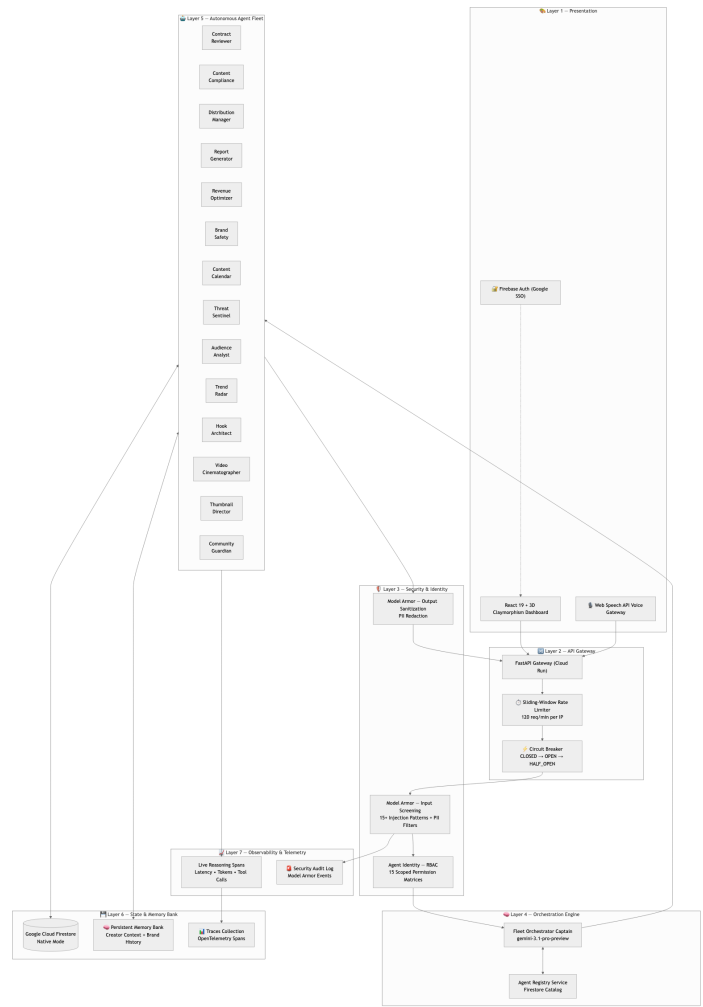
Complete 14-Diagram System Architecture & Engineering Blueprint

Target Track:	Track 3 — The Fortified Enterprise Fleet	Hackathon:	All Things Agentic Hackathon 2026
Live Deployment:	Google Cloud Run (us-central1)	GCP Project:	crewmate-507013
Architecture Pattern:	7-Layer Hexagonal GEAP Architecture	Engineers:	Lovejeet Singh & Sachit Babbar
Fleet Scale:	15 Autonomous Agents (ADK + Gemini)	GEAP Compliance:	7/7 Enterprise Components (100%)

1. Executive Architecture Summary

Crewmate is a production-grade 15-agent autonomous enterprise fleet that equips solo content creators with the same governance, security, and multi-agent intelligence that Fortune 500 enterprises rely on. Powered by **Google ADK**, **Gemini 3.7 Flash**, **Gemini 3.1 Pro Preview**, **Vertex AI**, **Google Veo 3.1**, **Imagen 3**, and **Google Cloud Firestore**, Crewmate implements the complete 7-pillar **Gemini Enterprise Agent Platform (GEAP)**.

2. High-Level 7-Layer GEAP System Architecture



3. Layer-by-Layer Deep Dive: Edge to Security

The system enforces strict unidirectional processing. Requests traverse Layer 1 (Presentation) to Layer 2 (API Gateway) and Layer 3 (Security & Identity) before reaching the Orchestration engine.

Diagram 2.1 — Layer 1: Presentation & Multimodal Interface

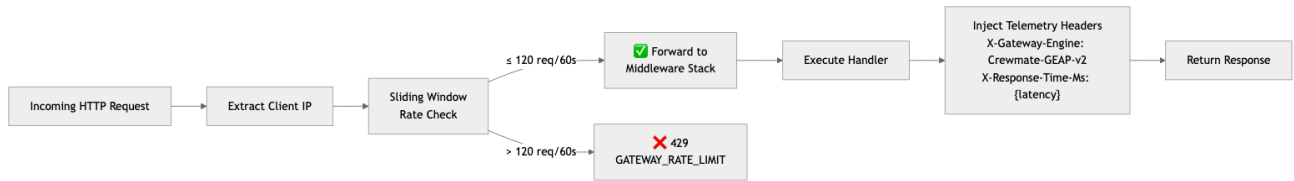


Diagram 2.2 — Layer 2: API Gateway & Traffic Governance

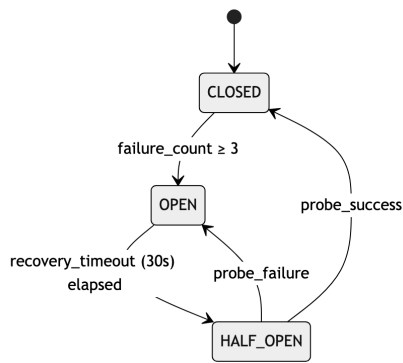
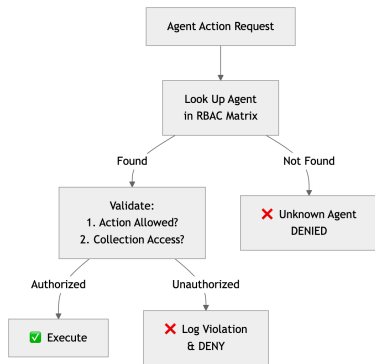


Diagram 2.3 — Layer 3: Security, Guardrails & Identity (Model Armor)



4. Orchestration Engine & Autonomous 15-Agent Fleet

The Fleet Orchestrator Captain decomposes high-level creator goals into specialized sub-tasks and delegates them across 14 worker agents organized into 6 functional domain clusters:

Diagram 3.1 — Layer 4: Orchestration Engine Architecture

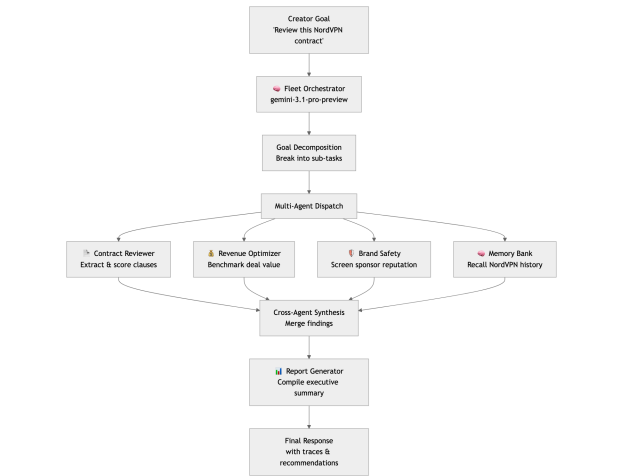
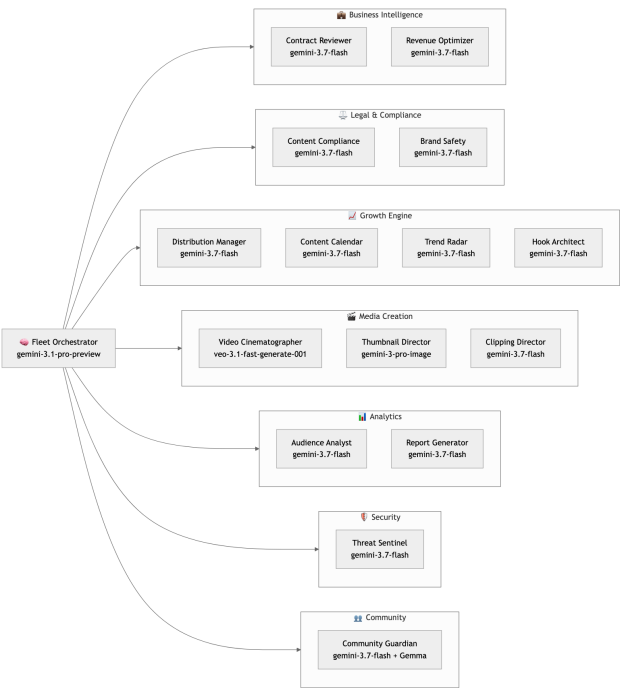


Diagram 3.2 — Layer 5: 15-Agent Fleet Hierarchy & Domain Clusters



5. State, Memory Bank & Distributed Observability

Layer 6 provides persistent cross-session memory for brand deal histories and creator preferences in Firestore. Layer 7 records OpenTelemetry spans tracking token usage, tool latencies, and security audit logs.

Diagram 4.1 — Layer 6: State & Memory Bank Flow

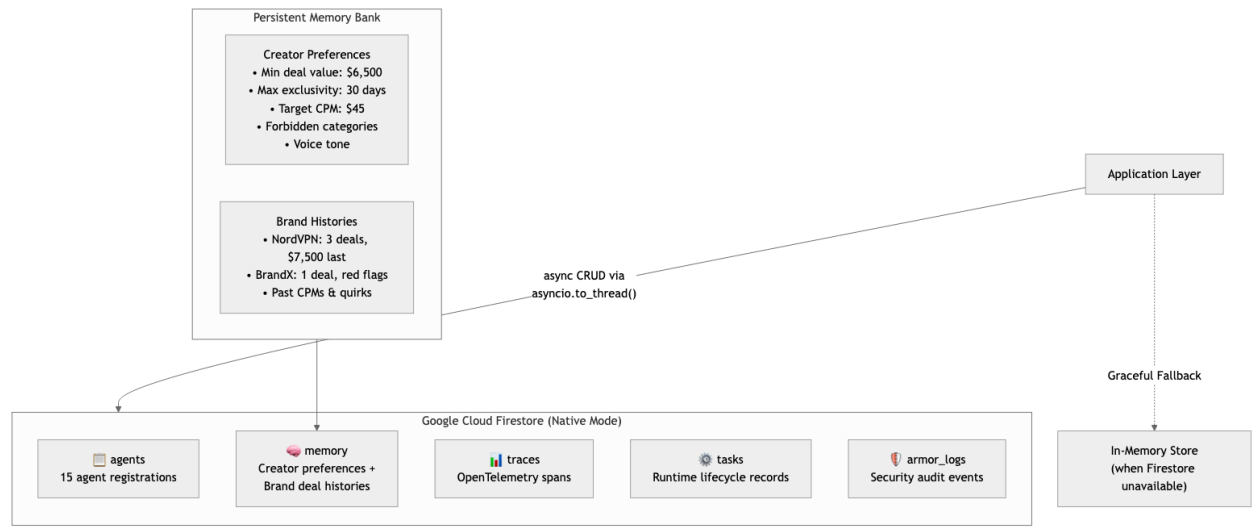


Diagram 4.2 — Layer 7: Observability & Telemetry Pipeline



6. End-to-End Autonomous Pipeline Sequence Flows

Demonstrating end-to-end execution across the 7 layers for Contract Review and Content Compliance:

Diagram 5.1 — Autonomous Contract Review Pipeline (Sequence Flow)

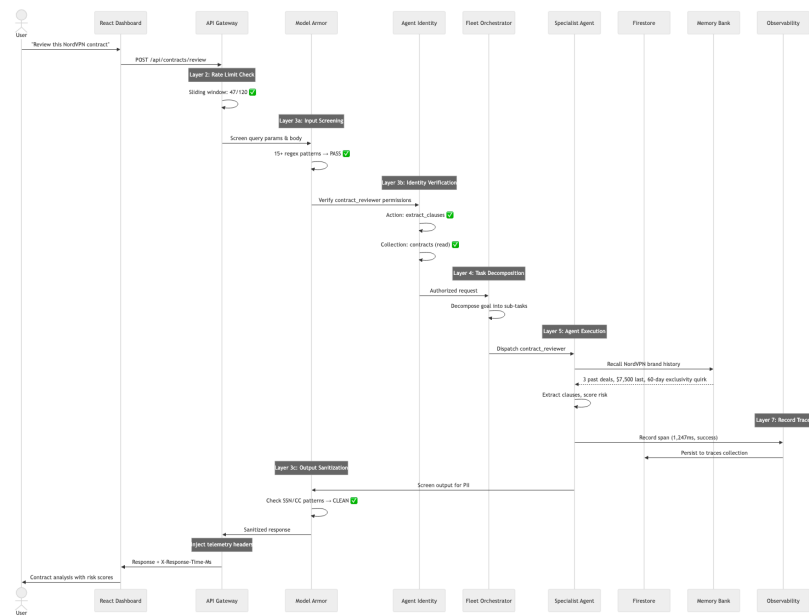


Diagram 5.2 — Content Compliance Scan & Lyria Audio Replacement



7. Multimodal Voice Gateway & Model Armor Guardrails

Voice commands pass through Web Speech API transcription before routing. Inbound and outbound payloads are protected by Model Armor:

Diagram 6.1 — Multimodal Voice Command Flow

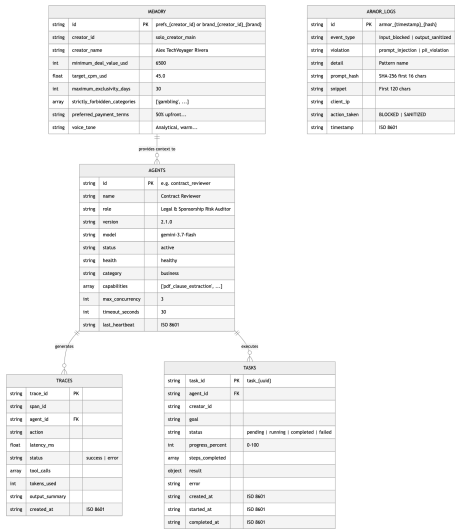
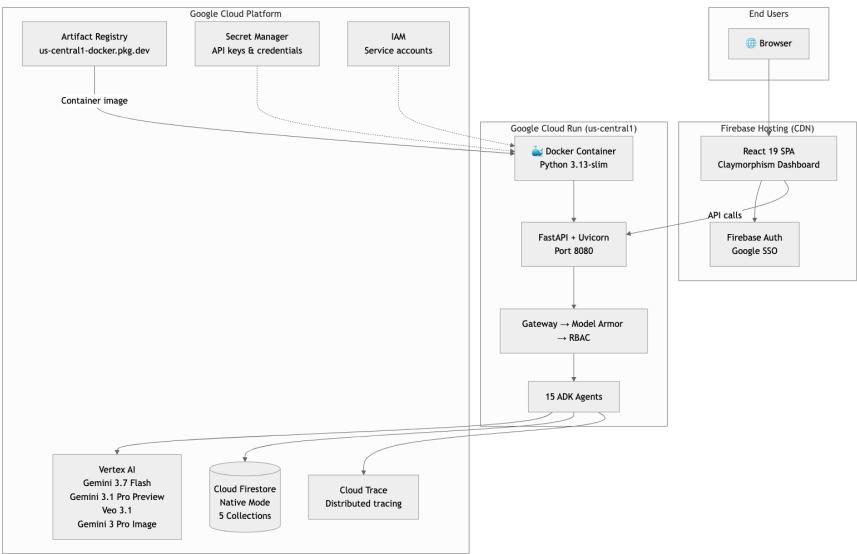


Diagram 6.2 — Model Armor Security Guardrails & Interception Flow



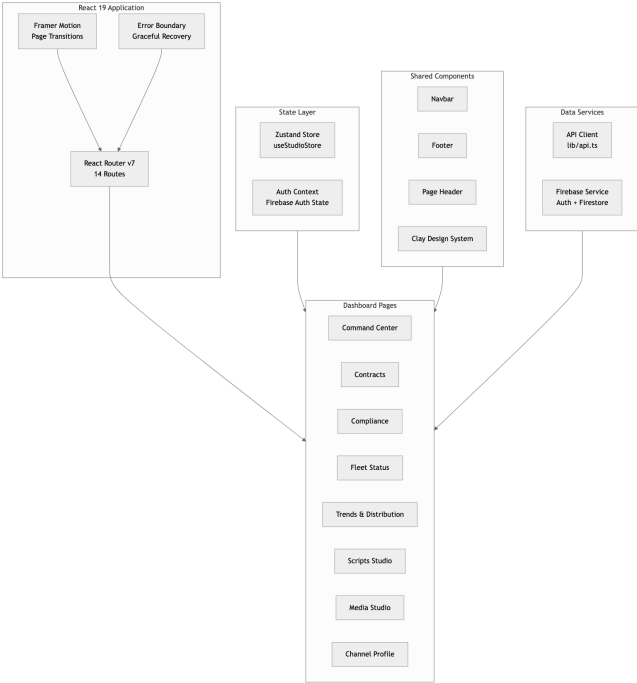
8. Circuit Breaker Resilience & Google Cloud Deployment

The platform guarantees resilience through 3-state circuit breaking and single-container serverless hosting on Google Cloud Run:

Diagram 7.1 — Circuit Breaker 3-State Finite State Machine



Diagram 7.2 — Google Cloud Run Production Deployment Topology



9. GEAP Component Matrix & Architecture Decision Records

#	GEAP Component	Implementation Details	Source File
1	Agent Registry	Firestore catalog with 15 agents, capability arrays, versions, and live health status.	services/registry.py
2	Memory Bank	Firestore persistent store for creator preferences, minimum deal rates, and brand deal logs.	services/memory.py
3	Model Armor	Pre-execution screening (15+ injection patterns, PII filters) + post-execution PII redaction.	middleware/model_armor.py
4	Agent Identity	Zero-trust RBAC matrix with scoped allowed_actions and collection read/write boundaries.	middleware/identity.py
5	Agent Gateway	FastAPI Gateway with sliding-window rate limiter (120 req/min) and 3-state circuit breaker.	middleware/gateway.py
6	Observability	OpenTelemetry-compliant SpanContext tracking latency, tokens, and tool call metrics.	services/observability.py
7	Agent Runtime	Async lifecycle state machine: pending -> running -> completed/failed with progress tracking.	services/runtime.py

Key Architecture Decision Records (ADRs)

- **ADR-01: Hierarchical Supervisor vs. Peer Swarm:** Selected Supervisor Orchestrator (gemini-3.1-pro-preview) to prevent infinite loops, enforce deterministic token budgets, and guarantee RBAC before tool execution.
- **ADR-02: Cloud Firestore (Native Mode) vs. Cloud SQL:** Selected Firestore for zero-schema migration overhead, dynamic JSON agent states, and native real-time document listeners for live UI telemetry.
- **ADR-03: Model Armor Middleware vs. LLM-as-a-Judge:** Built deterministic regex and heuristic Model Armor middleware to reduce security latency from ~800ms to <10ms and prevent recursive prompt injections.
- **ADR-04: Single-Stage Unified Container Deployment:** Unified FastAPI backend and React 19 static SPA into a single Python 3.13-slim Cloud Run container, eliminating multi-service networking latency and CORS friction.

Crewmate Engineering Team: Lovejeet Singh (Engineering Lead) & Sachit Babbar (Research & Docs Lead)
Live Production URL: <https://crewmate-285381944529.us-central1.run.app> • Repository: <https://github.com/z-lovejeet/Crewmate>